

Topic 03: Silent Failure Prevention: Pytest & Quality

Basic Software Engineering Workflows

TOPIC 03

Silent Failure Prevention: Pytest & Quality

Deterministic unit testing, test fixtures, parametrization, and ruff linting.

Cognitive Objectives

By the end of this topic, learners will be able to:

- **Prevent silent data corruption** in ML feature pipelines using deterministic unit tests.
- **Structure modular test fixtures** and manage setup/teardown lifecycles using dependency injection.
- **Scale test coverage systematically** with `@pytest.mark.parametrize` across critical edge cases.
- **Automate code hygiene** using ultra-fast static linters (`ruff`) and type checkers (`mypy`).

The Silent Failure Disaster

The Hazard: Code Runs, Data Dies

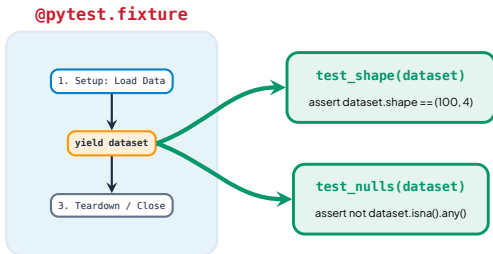
- A feature normalization function contains an off-by-one division error or silently fills missing values with 0.0.
- The script exits with exit code 0 without raising any exceptions!
- Model trains on corrupted features and deploys to production.
- **Business Impact:** Millions in mispriced loans before anyone notices!

The "Aha!" Discovery: Executable Contracts

- Unit tests act as **executable specifications** that verify data transformations across known invariants.
- Test for shape preservation, non-negativity, mathematical bounds, and null rejection.
- Defensive assertions catch drift before weights are fitted.

PUNCHLINE Crashing early is a feature; silent data corruption is a disaster.

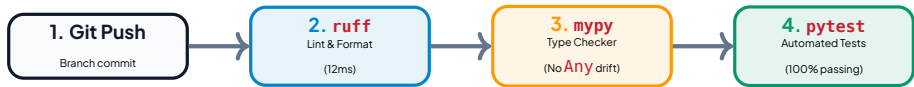
Hero Visualization: Fixture Lifecycle & Injection



Dependency Injection Mechanics

- **Auto-Wiring:** Pytest passes fixture return values into test functions whose parameter names match the fixture name.
- **Deterministic Isolation:** Each test function receives a fresh copy of the resource.
- **Scopes:** `function` (default), `module` (once per file), `session` (once per test run).

Hero Visualization: Modern CI/CD Quality Gate Pipeline



PUNCHLINE

Automated pre-commit hooks and CI pipelines stop defective code before it reaches production.

Production Checkpoint: Parametrized Test Suite

Parametrization Invariants

- Automatically executes the same test across multiple input/output tuples.
- Tests nominal inputs alongside zero vectors, empty matrices, and NaN edge cases.
- Generates individual test reports for each parameterized test case.

PUNCHLINE Edge case testing eliminates production surprise failures.

Parametrized Test Code

```
1 import pytest
2 import numpy as np
3
4 @pytest.mark.parametrize("vec, expected_norm", [
5     (np.array([3.0, 4.0]), 5.0),
6     (np.array([0.0, 0.0]), 0.0),
7     (np.array([1.0, 1.0, 1.0]), np.sqrt(3)),
8 ])
9 def test_vector_norm_calculation(vec, expected_norm):
10     result = np.linalg.norm(vec)
11     assert np.isclose(result, expected_norm, atol=1e-6)
```